

5 分布式存储

冯奕
2026



南京大學
NANJING UNIVERSITY

目录

- 分布式存储系统
 - 数据和系统类型
 - 分布式文件系统
 - 从单机到分布式—具体如何分布式
- 案例1：区块链
- 案例2：MongoDB
- 案例3：Cassandra
- 案例3：Ceph



分布式存储系统—类型

- 云计算中分布式计算主要作为“PaaS”，分布式存储则作为IaaS、PaaS、SaaS都有
- 根据存储的数据类型（结构化、非结构化、半结构化等），将存储系统分为以下几类
 - 分布式文件系统——泛指以分布式的方式存储文件的系统，文件可以多种形式存在
 - 三种数据类型：Binary large object(Blob)二进制大对象，定长块，大文件
 - 提供不同类型的存储服务：对象存储、文件存储、块存储
 - 可以作为分布式键值存储、分布式表、分布式数据的底层存储。GFS，弹性块存储EBS，Ceph
 - 分布式键值系统
 - 用来存储关系简单的半结构化数据，提供基于主键的CRUD功能
 - 可以看作是对分布式表的简化，一般用来作缓存，例如Memcached。一致性Hash是常用技术
 - 分布式表
 - 用于存储半结构化数据；以表格为单位组织数据，一个表格有多行，通过主键标识一行
 - 不仅仅支持简单的CRUD，还支持扫描某个主键的范围和范围查找功能。Google Bigtable
 - 分布式数据库
 - 基于传统关系型数据库发展而来，例如分布式MySQL



分布式存储系统—文件系统的发展

- 狭义上的“文件系统”——不同于分布式文件存储中的块存储、对象存储
- 单机文件系统
 - 核心是使用树型数据结构组织文件、目录以及访问控制；随着单机能够管理的存储空间在变大，提升管理能力和性能的文件系统也在演化。
- 网络文件系统
 - 目标是让用户能够以访问本地文件系统的方式访问远程机器上的文件，提供跨平台的文件共享系统
- 并行文件系统
 - 用在大规模并行处理体系结构中，保证一个业务的多个并行任务可以同时为同一个文件的不同位置并行处理
- 分布式文件系统
 - 采用集中式管理、分布式存储的架构，将文件实际存储在多个不同的节点上，且每一个部分都有多个副本
- 高通量文件系统
 - 专指为大型数据中心设计的分布式文件系统，将数据中心所有的低成本存储资源有效地组织起来服务于上层多种应用的数据存储需求和数据访问需求



分布式存储系统——从单机到分布式

- 单机存储系统

- 硬件基础
- 存储引擎：实现数据的基本操作，包括增删改查，读取分随机读和顺序扫描，重点是数据结构
- 数据模型：存储系统的外壳，三类数据模型：文件、关系、键值；对应文件系统、数据库、键值存储

- 分布式存储系统

- 对单机的扩展，面临的重要问题是：

- 如何将数据均匀的分布到多个存储节点
- 如何保证提供高可用性的数据多副本始终保持一致
- 如何检测节点故障并高效应对

- 评价指标

- 性能：吞吐率-在某一段时间可以处理的请求总数；系统响应时间-从某个请求发出到收到结果的时间
- 可用性：指在系统面对各种异常时可以提供的正常服务能力；用停止服务的时间和正常时间比重度量
- 一致性：越强的一致性模型用户使用起来越简单——可能牺牲可用性或分区容错性
- 可扩展性：能否通过增加服务器数量提高系统能力或者增加服务器的难度；理想的“线性可扩展”

性能分析-性能优化
负载均衡
数据复制
故障检测



衍生的相关工作



目录

- 分布式存储系统
 - 数据和系统类型
 - 分布式文件系统
 - 从单机到分布式—具体如何分布式
- 案例1：区块链
- 案例2：MongoDB
- 案例3：Cassandra
- 案例3：Ceph



区块链——定义

- 现实需求

- 一个共享的、不可篡改的账本，用于记录交易或者事件；所有人都会记录一份
- 每当要持久化新的记录时，需要大多数人都同意，否则无法记录

- 区块链

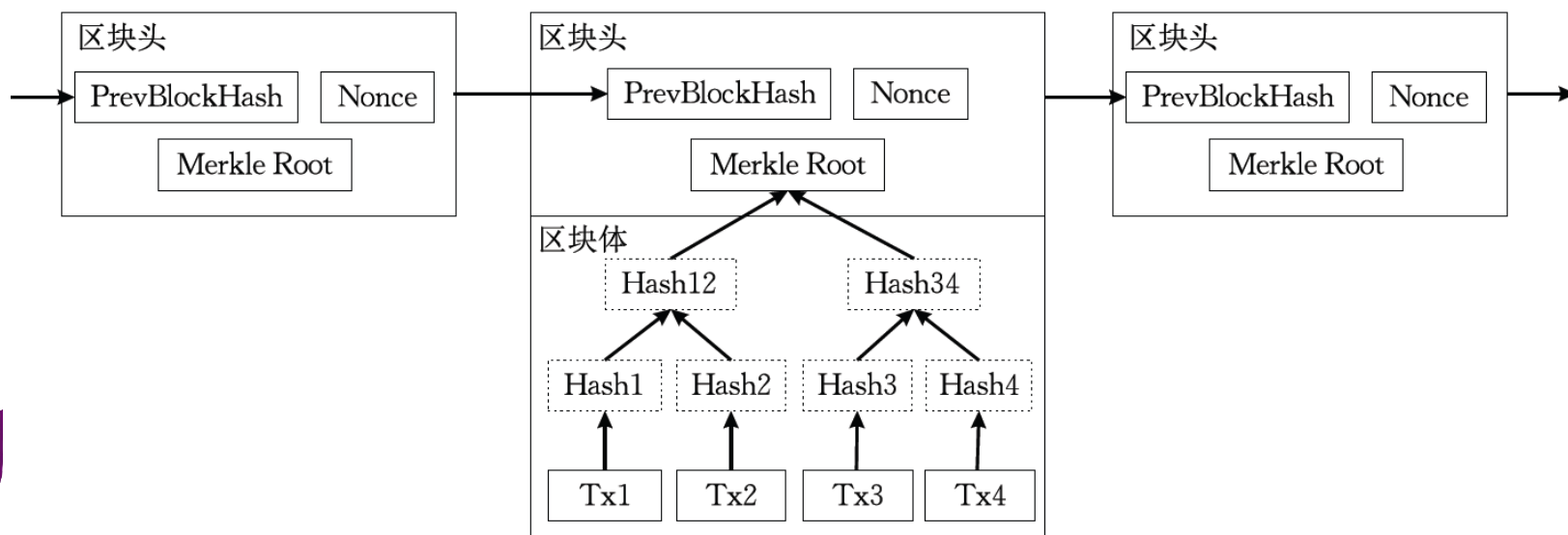
- **IBM定义**：区块链是一个共享的、不可篡改的账本，用于促进业务网络中的交易记录和资产跟踪。这种账本可以跟踪有形资产（如房屋、汽车、现金、土地）和无形资产（如知识产权、专利、版权、品牌）。区块链是安全共享的去中心化的数据账本，支持一组特定的参与方共享数据，并结合了分布式数据存储、点对点传输、共识机制、加密算法等计算机技术。
- **维基百科定义**：区块链是一个借助密码学与共识机制等技术建立和储存庞大交易数据链的点对点网络系统。每个区块包含了前一个区块的加密散列、时间戳记及交易数据，这些设计使得区块内容具有难以篡改的特性。
- **百度百科定义**：狭义区块链是按照时间顺序，将数据区块以顺序相连的方式组合成的链式数据结构，并以密码学方式保证的不可篡改和不可伪造的分布式账本。广义区块链技术是利用块链式数据结构验证与存储数据，利用分布式节点共识算法生成和更新数据，利用密码学的方式保证数据传输和访问的安全、利用由自动化脚本代码组成的智能合约，编程和操作数据的全新的分布式基础架构与计算范式。



区块链——重要概念（1）

• 重要概念

- **区块** (Block)：区块是区块链中的基本单位，它包含了一定数量的交易数据和其他元数据，如时间戳和区块哈希值。
- **链** (Chain)：区块通过哈希值链接在一起形成一个链，每个区块都包含了前一个区块的哈希值，确保了数据的连续性和完整性
- **去中心化** (Decentralization)：区块链没有中心化的控制机构，区块链技术通过分布式网络结构实现去中心化，不依赖于任何单一实体进行数据的管理和控制，数据被存储在网络中的多个节点上，每个节点都有权访问整个数据库，所有的参与者共同维护和验证数据的完整性，消除了单点故障和信任问题。



区块链——重要概念 (2)

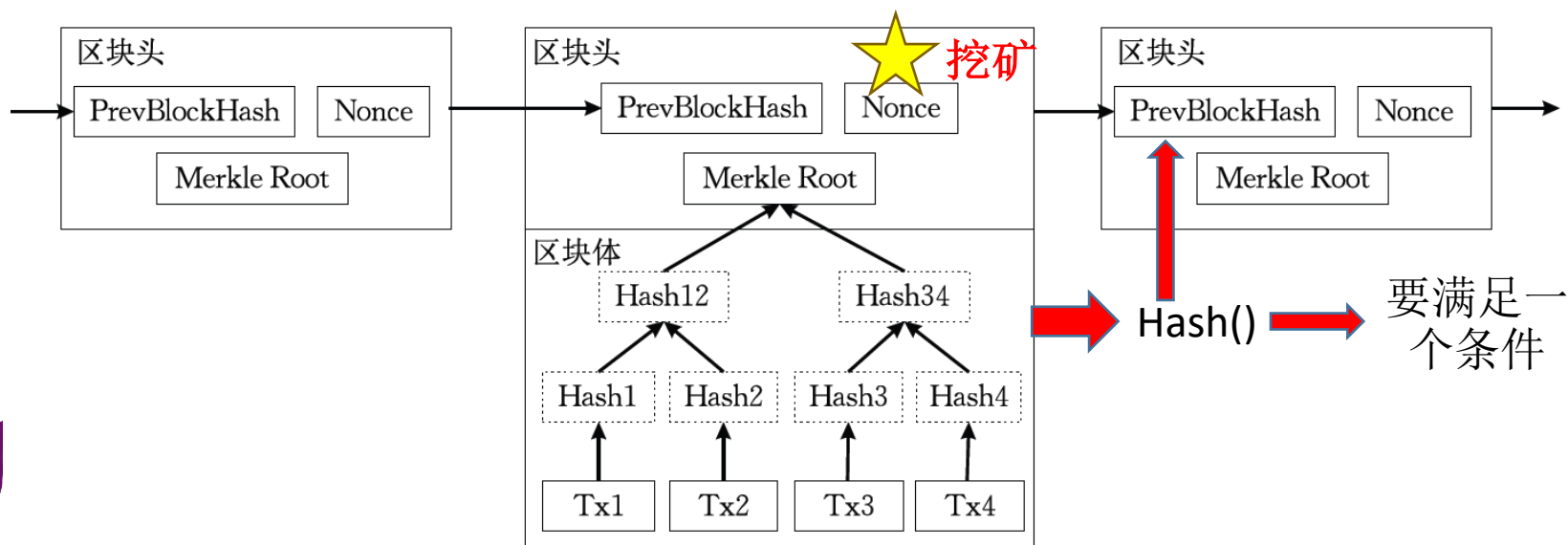
- 重要概念

- **共识机制** (Consensus Mechanism)：区块链通过共识算法来解决分布式环境下的数据一致性问题，确保了网络中的所有参与者在没有中心权威的情况下，可以就数据的正确性达成一致。常见的共识机制包括工作量证明 (PoW, Proof of Work) 和权益证明 (PoS, Proof of Stake) 等。

称为难度值，随着区块增加而降低

- **PoW**：找到一个Nonce让BlockHash满足一个条件，例如前面有固定数目个的0。

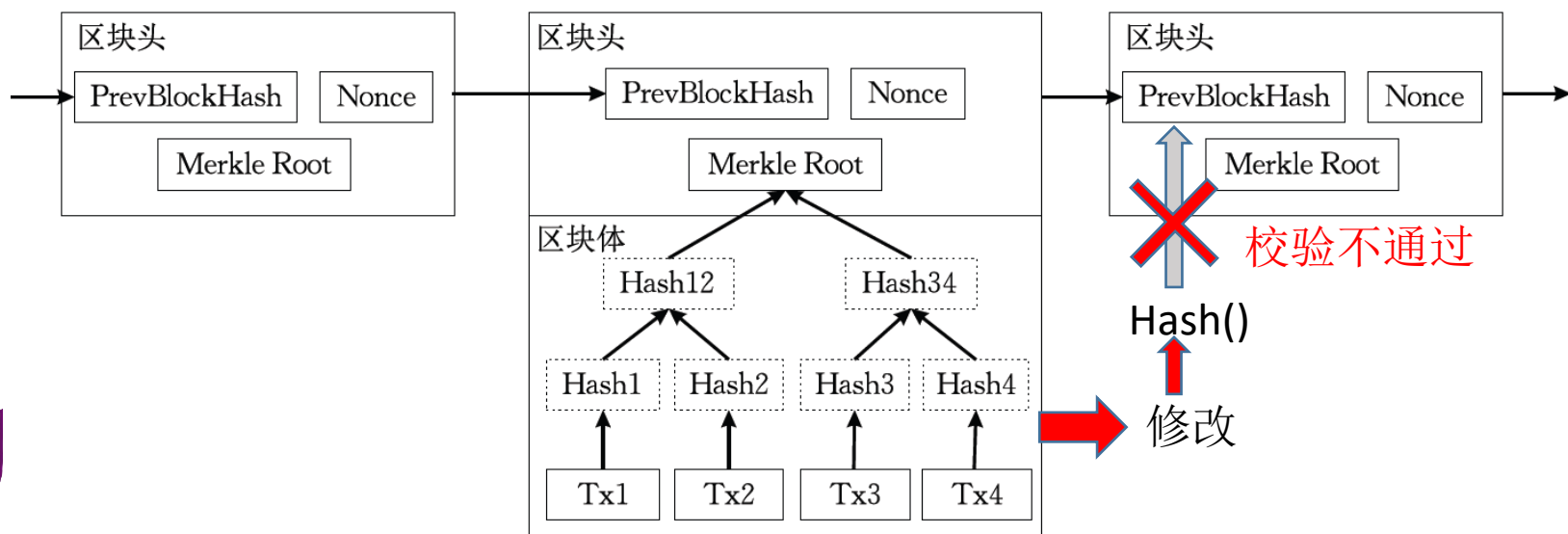
$\text{PrevBlockHash} = \text{Hash}(\text{index}, \text{previous block hash}, \text{timestamp}, \text{block data}, \text{nonce})$



区块链——重要概念（3）

• 重要概念

- **分布式账本** (Distributed Ledger)：区块链中的数据副本分布在网络的多个节点上，每个节点都有完整的账本副本，确保了数据的可靠性和安全性。
- **不可篡改性** (Immutability)：由于区块链的结构和分布式特性，一旦数据被添加到区块链上，它就会永久记录下来，难以丢失、损坏或修改。这是因为每个区块都包含了前一个区块的哈希值，任何对区块数据的修改都会导致哈希值不匹配，从而被网络中的其他节点检测出来。。



区块链——重要概念（4）

- 重要概念

- **透明性** (Transparency)：区块链的交易记录对所有网络参与者开放，从而提高了系统的透明度。虽然交易记录是公开的，但通过加密技术，交易双方的身份信息可以保持匿名。
- **可编程性** (Programmability)：许多现代区块链支持智能合约，这是自动执行的程序，可以在预定条件满足时触发特定的行为，增强了安全性和可靠性、提高了效率以及降低了成本。可编程性是构建去中心化应用的基础，扩展了区块链的应用范围。
- **结论**：区块链是一种特殊的分布式数据库，和传统数据库有着显著的区别。除了存储和访问数据，区块链还包括一种机制，这种机制利用加密技术和共识算法来验证和保护数据的完整性。这是传统数据库不具备的特性，也正是区块链技术的核心创新之处。
- **缺点**：存储开销、计算开销太大，不利于环境保护！



区块链——工作过程

- **创建交易** (Transaction Creation) : 在区块链中, 一切都从交易开始。
- **交易验证** (Transaction Verification) : 交易完成后, 它需要被网络中的其他节点 (计算机) 验证。这些节点会检查交易是否有效。
- **创建区块** (Block Formation) : 一旦交易被验证, 它就会被放入一个待处理的区块中。这个区块还包含其他的信息, 比如时间戳和引用到前一个区块的哈希值。
- **挖矿或创建新的区块** (Mining or New Block Creation) : 接下来的步骤是“挖矿”, 这是一个解决复杂数学问题的过程。首先解决这个问题的节点将会创建新的区块并将其添加到区块链中。
- **将新区块添加到链上** (Adding the New Block to the Chain) : 新的区块一旦被网络接受, 它就会被添加到区块链的末端, 并且任何在该区块中的交易就被认为是被确认的。
- **完成交易并重复过程** (Completion of Transaction and Repetition of Process) : 交易完成后, 新的交易发生, 整个过程将从步骤1开始重新进行。



区块链——比特币

- 基于区块链的关系
 - 用区块链存储交易记录——只记录账户变化，即增加、减少
 - 用区块的计算作为产生比特币的时机——每一个新的区块产生时，会创建新的比特币；只有计算出适合当时的Nonce的人才能获得奖励
 - 所有的交易，包括产生奖励的交易，都会被所有人记录下来
 - 交易由某个节点发起，发送给其他人
 - 每一个人收到一个新的交易的时候，会首先验证是否符合条件，比如交易双方余额是否充足等



目录

- 分布式存储系统
 - 数据和系统类型
 - 分布式文件系统
 - 从单机到分布式—具体如何分布式
- 案例1：区块链
- 案例2：MongoDB
- 案例3：Cassandra
- 案例3：Ceph



MongoDB——简介 (1)



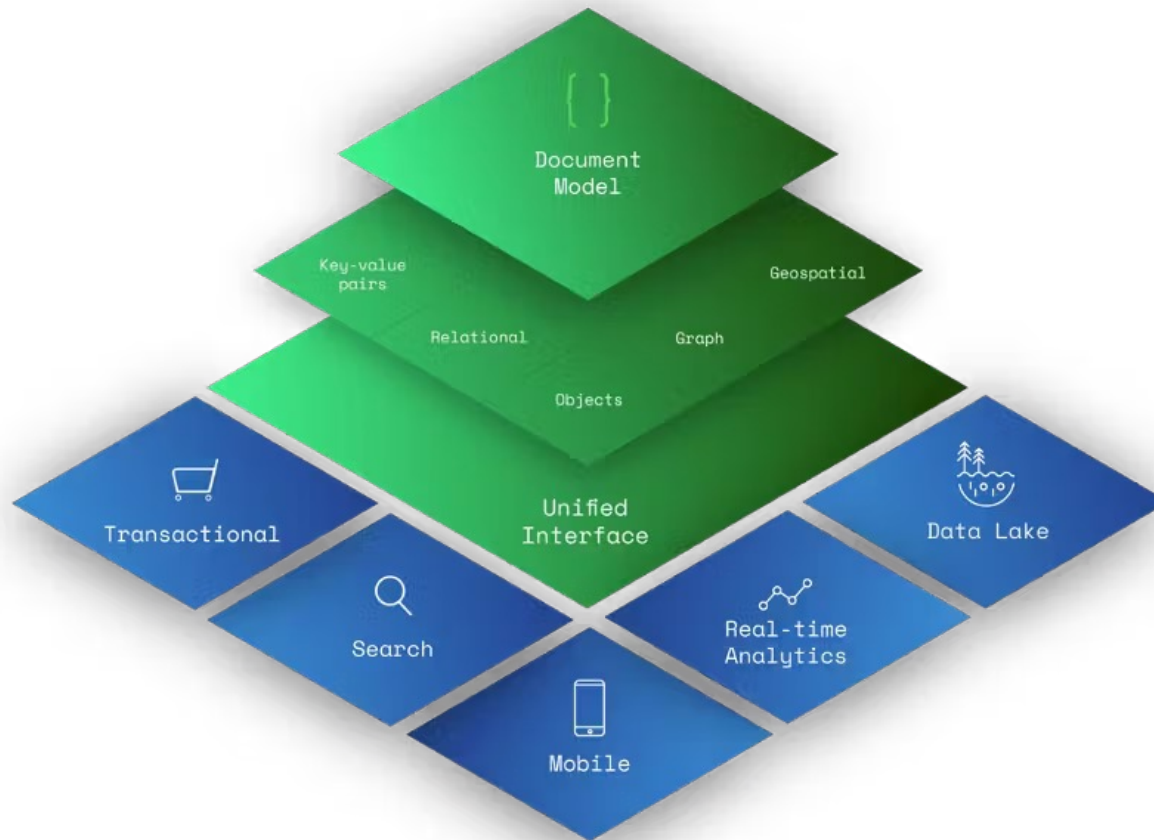
- MongoDB是一个基于**分布式文件存储**的数据库。
- 由C++语言编写。旨在为WEB应用提供**可扩展**的高性能数据存储解决方案。
- 是一个介于关系数据库和非关系数据库之间的产品，是非关系数据库当中功能最丰富，最像关系数据库的。
- Mongo最大的特点：支持的查询语言非常强大，其语法有点类似于面向对象的查询语言，几乎可以实现类似关系数据库单表查询的绝大部分功能，而且还支持对数据建立索引。
- 易使用
 - 面向文档的数据库，没有行的概念，支持嵌入文档和数组，没有预定义的表结构
- 易扩展
 - 纵向扩展：增强服务器能力
 - 横向扩展：增加服务器数量；数据很容易在多台服务器间分割；自动处理跨集群的数据和负载；准确路由用户请求等



MongoDB——简介 (2)

The document model is a superset of other data models

Due to their rich data modeling capabilities, document databases are general-purpose databases that can store data for a variety of use cases.



MongoDB——简介 (3)

- 支持多种开发语言
- 索引Indexing
 - 通用二级索引、唯一索引、复合索引
 - 地理空间索引、全文索引等

Getting Started

MongoDB provides a [Getting Started Guide](#) in the following editions.

[mongo Shell Edition](#)

[Python Edition](#)

[Java Edition](#)

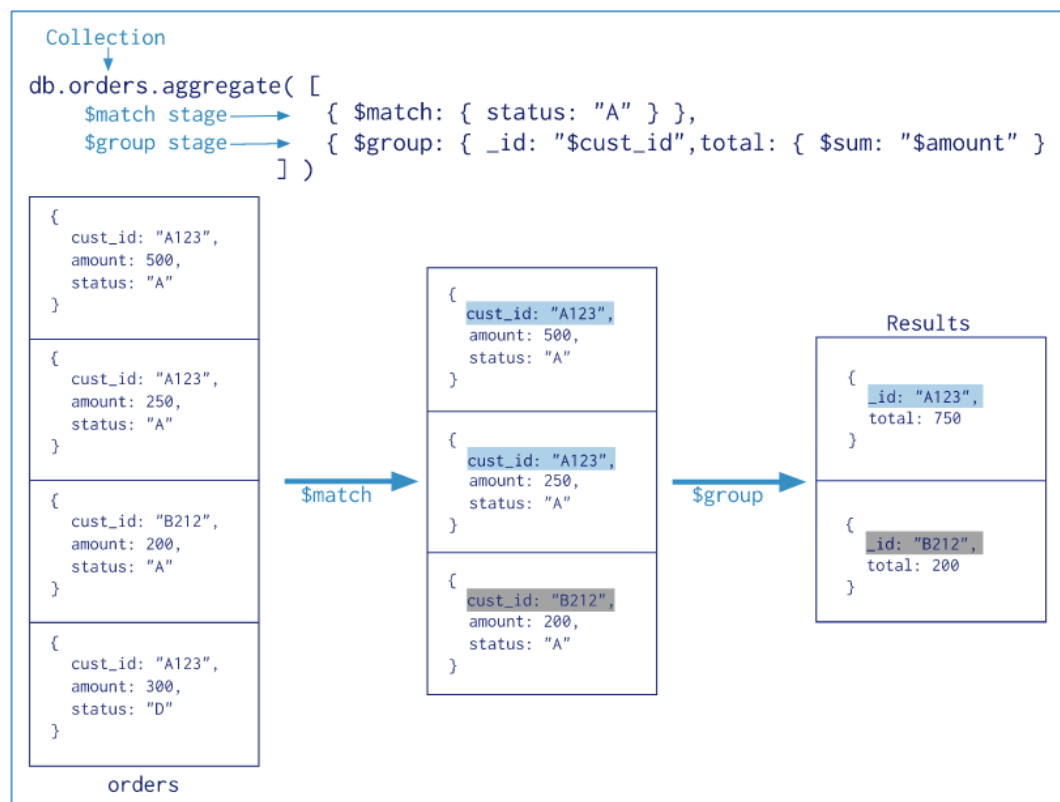
[Ruby Edition](#)

[Node.JS Edition](#)

[C++ Edition](#)

[C# Edition](#)

- 聚合Aggregation
 - 聚合管道、Map-Reduce
- 特殊的集合类型
 - 例如存在时间有限的集合
- 文件存储
 - 支持存储大文件和文件元数据



MongoDB——文档

- 文档
 - 键值对的有序集
 - {key1:value1, key2:value2, key3:value3,..., keyn:valuen}
 - value可以是任意数据类型，包括数组和文档
 - 同一个文档中key不能重复
- 使用文档的优点
 - 文档是很多编程语言中原生支持的数据类型
 - 可嵌入的文档和数组避免了很多费时的Join操作
 - 灵活的schema支持非常顺畅的多态性

```
{  
  name: "sue",  
  age: 26,  
  status: "A",  
  groups: [ "news", "sports" ]  
}
```

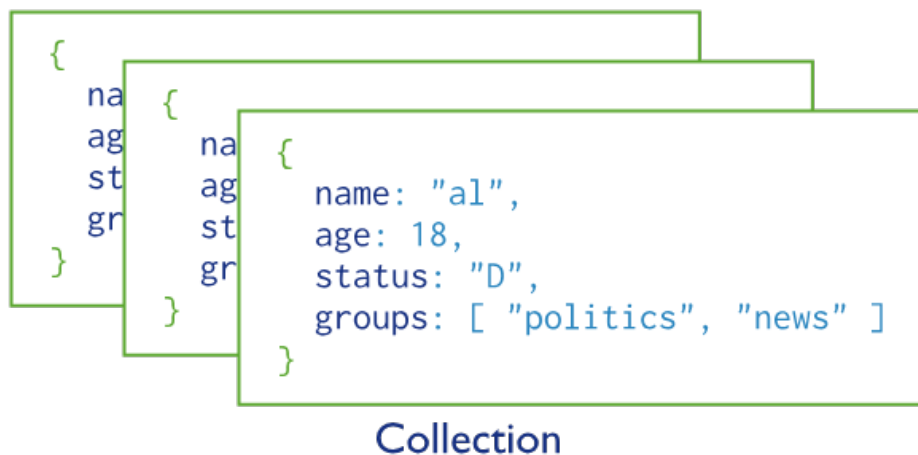
← field: value
← field: value
← field: value
← field: value



MongoDB——集合与数据库

- MongoDB中的文档存储在Collection中
 - 一组文档就是一个集合；类似关系型数据库中的“表”
 - 同一个集合中的文档不一定是相同的格式；使用的时候最好是相同的

- 数据库Database
 - 多个集合构成了数据库



- 文档的键_id
 - 对应的键值在同一个集合中是唯一的
 - 可以是系统自动生成的ObjectId，也可以是用户指定的
- MongoDB以BSON documents格式存储数据
 - BSON是JSON的二进制表示，但是比JSON支持更多的数据类型



MongoDB——文档、集合示例

数据类型包括
null
boolean
数值
浮点数或者整型
字符串
日期
正则表达式
数组
内嵌文档
对象id
二进制数据
代码

```
{
  "_id": 1,
  "first_name": "Tom",
  "email": "tom@example.com",
  "cell": "765-555-5555",
  "likes": [
    "fashion",
    "spas",
    "shopping"
  ],
  "businesses": [
    {
      "name": "Entertainment 1080",
      "partner": "Jean",
      "status": "Bankrupt",
      "date_founded": {
        "$date": "2012-05-19T04:00:00Z"
      }
    },
    {
      "name": "Swag for Tweens",
      "date_founded": {
        "$date": "2012-11-01T04:00:00Z"
      }
    }
  ]
}
```

```
{
  "_id": 2,
  "first_name": "Donna",
  "email": "donna@example.com",
  "spouse": "Joe",
  "likes": [
    "spas",
    "shopping",
    "live tweeting"
  ],
  "businesses": [
    {
      "name": "Castle Realty",
      "status": "Thriving",
      "date_founded": {
        "$date": "2013-11-21T04:00:00Z"
      }
    }
  ]
}
```



MongoDB——基本操作（1）

- 创建数据库、创建集合

- 如果数据库、集合不存在，只有第一次写入数据的时候才创建

- use myNewDB

- db.myNewCollection1.insertOne({ x: 1 })

- 可以显式地创建集合

- db.createCollection()

- 文档的CRUD操作

- Create:

- db.collection.insertOne()

- db.collection.insertMany()

- db.collection.insert()

- 当更新或者查询并修改等操作发现文档不存在时，会插入新文档

- Read: db.collection.find()等

- Update: db.collection.updateOne(); updateMany(); replaceOne()

- Delete: db.collection.deleteOne(); deleteMany()

Insert a Single Document

`com.mongodb.reactivestreams.client.MongoCollection.insertOne` inserts a *single document* into a collection with the *Java Reactive Streams Driver*:

```
Java (Async)
{ item: "canvas", qty: 100, tags: ["cotton"], size: { h: 28, w: 35.5, uom: "cm" } }
```

The following example inserts the document above into the `inventory` collection. If the document does not specify an `_id` field, the driver adds the `_id` field with an `ObjectId` value to the new document. See [Insert Behavior](#).

```
Java (Async)
Document canvas = new Document("item", "canvas")
    .append("qty", 100)
    .append("tags", singletonList("cotton"));
```

Select your language

- Java (Async)
- MongoDB Shell
- Compass
- C#
- Go
- Java (Async)
- Java (Sync)
- Motor
- Node.js
- Perl



MongoDB——基本操作 (2)

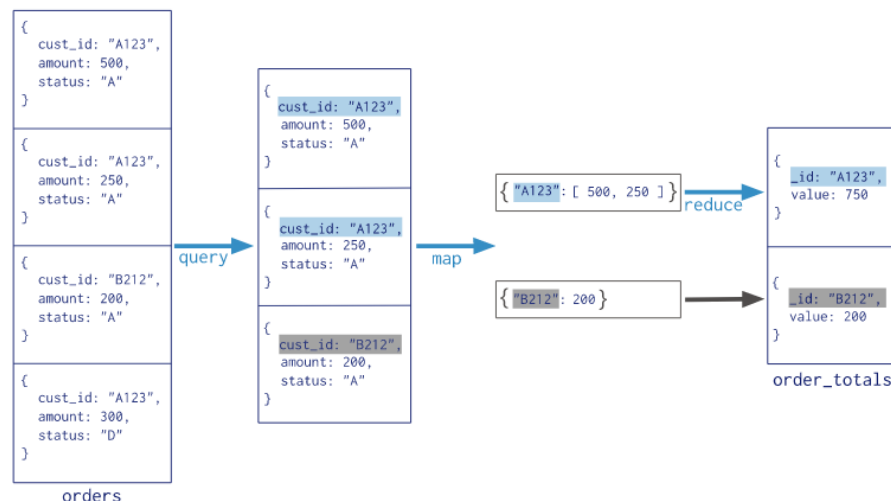
• 聚合

- 聚合管道 (各种操作)
- MapReduce:
 - 在满足query条件的输入上Map

```
Collection
  ↓
db.orders.mapReduce(
  map   → function() { emit( this.cust_id, this.amount ); },
  reduce → function(key, values) { return Array.sum( values ); },
  query → { status: "A" },
  output → "order_totals"
)
```

• 文本搜索

- 对字符串值进行索引
- 在字符串的索引上搜索
- 步骤:
 - 建立索引createIndex({k:v})
 - 使用\$text和\$search操作查询
 - `db.stores.find( { $text: { $search: "java coffee shop" } } )`
 - 还可以使用sort()方法和\$meta操作对结果进行排序
 - `{ score: { $meta: "textScore" } }).sort( { score: { $meta: "textScore" } } )`



MongoDB——索引 (1)

- 通用二级索引
 - 索引必须依赖一个key
 - Dense Index(指向非排序全局数据); Sparse Index(指向排序的Dense Index)
 - Cluster Index(将全局数据按照primary key排序后使用的Sparse Index); Secondary Index(指向非primary key列的Dense Index的Sparse Index)
- 复合索引
 - 在多个key上建立一个索引
- 索引对象和数组
 - 对嵌套文档的内容建立索引、对数组里每个元素建立索引
- 唯一索引
 - 文档在索引指定键的值具有唯一性
- 全文索引



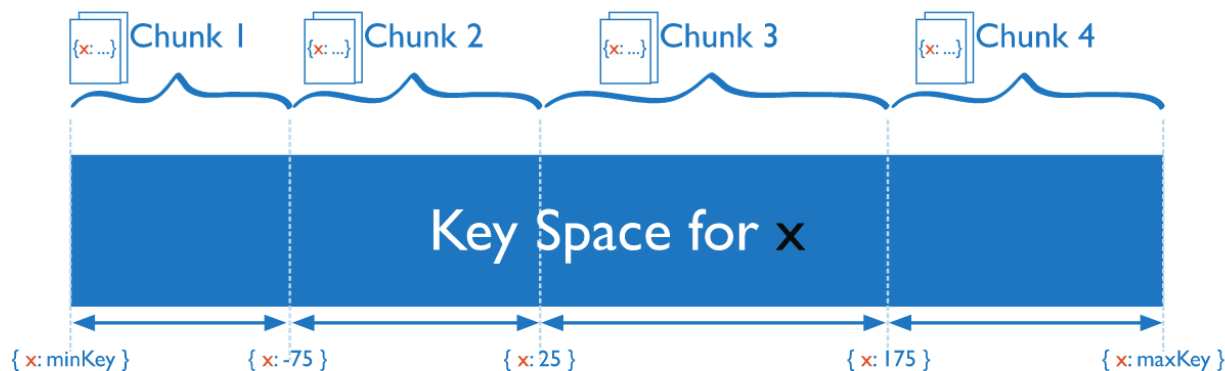
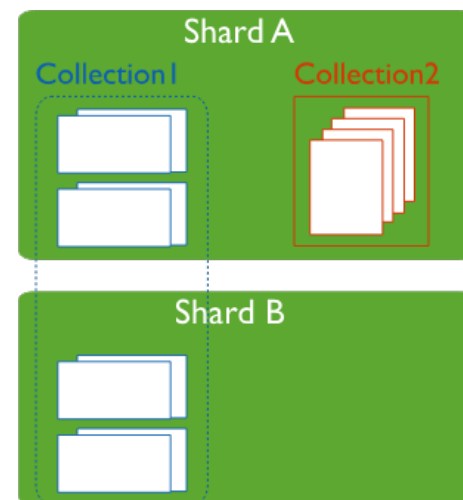
MongoDB——索引 (2)

- 地理空间索引：地理位置信息保存为GeoJSON或者legacy coordinate pairs中
- GeoJSON Object
 - `<field>: { type: <GeoJSON type> , coordinates: <coordinates> }`
 - 2dsphere索引
 - `db.collection.createIndex( { <location field> : "2dsphere" } )`
- Legacy coordinate pairs 坐标对
 - `<field>: [ <x>, <y> ]`
 - `<field>: { <field1>: <x>, <field2>: <y> }`
 - 2d索引
 - `db.collection.createIndex( { <location field> : "2d" } )`
- 一系列基于地理空间索引的查询操作
 - `$geoWithin`
 - `$near`
 - `$nearSphere`等



MongoDB——分片 (1)

- 分片Shard是MongoDB用来将大型集合分割到不同服务器上采用的方法
- 一个集合可以被分为多个分片
- 一个分片存储在一台服务器上
 - 一个分片可以有多个副本，副本存在其他服务器上
 - 存储同一个分片的多台服务器称为副本集
- 用来对集合进行分片的键称为“片键”
- 一个分片就是集合数据中片键在
 - 某个区间上的数据子集：ranged sharding; (还有Hashed Sharding)

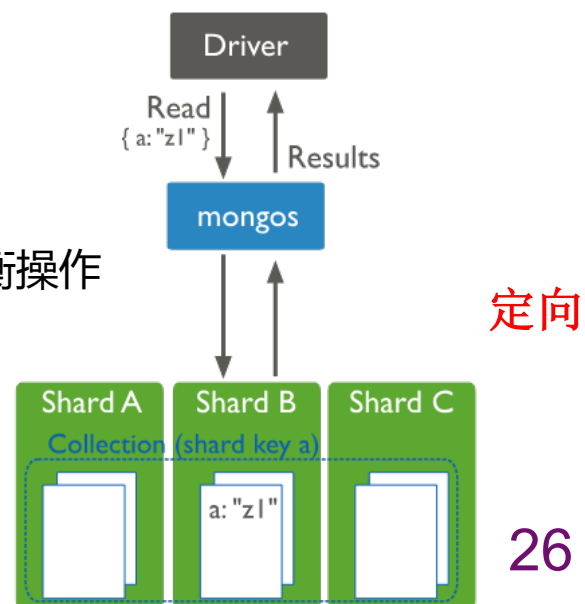
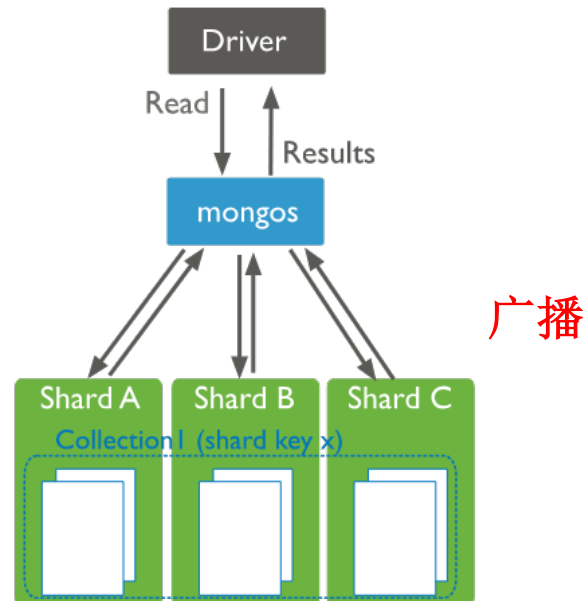


MongoDB——分片 (2)

- 一个分片包含多个片键区间
 - 每个区间称为数据“块”
 - 当块变得越来越大时，会将其分割成两个较小的块
 - 设置块的大小：默认200M

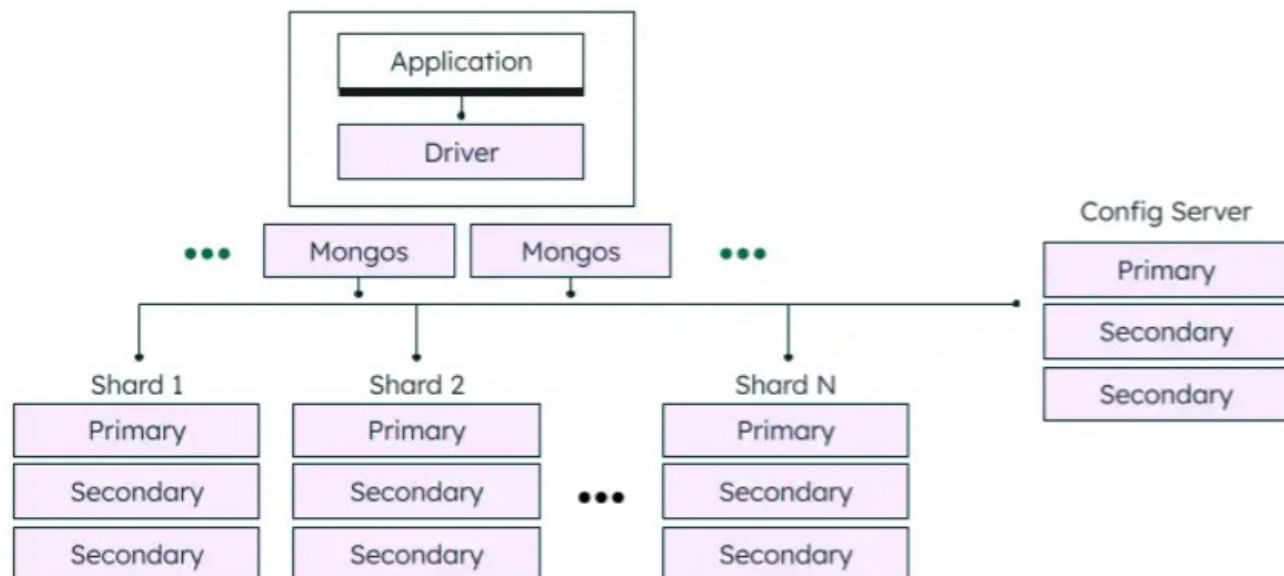
- 分片平衡

- 通过移动块使得分片的数据量得到平衡
- 新加一个分片服务器
 - 已有分片上的数据都移动一部分到新分片上
- 原有分片平衡
 - 如，当一个分片比另一个分片多出9个块，触发平衡操作



MongoDB——建立和启动集群

- 配置服务器 mongod
 - “1个或3个”；测试环境使用1个就够了
 - 配置服务器之间交互比较复杂，生产环境使用3个比较适合
- mongos进程
 - 隐藏集群内分片的复杂性，向用户提供简洁的单服务器接口
- 分片
 - 选择片键
 - 数据分片
 - 新增分片
 - 移除分片



MongoDB——选择片键 (1)

- 可能不是好的片键的例子
 - 小基数片键
 - 利用取值很少的字段作为片键，如性别、民族等
 - 高频值片键
 - 片键对应的取值中存在某些值频率非常高的情况

{ X : 1 } { X : 2 } { X : 1 }

Shard A

Shard B

Shard C

$\text{minKey} \leq X < 1$

$1 \leq X < 2$

$2 \leq X < \text{maxKey}$

{ X : 12 } { X : 12 } { X : 12 } { X : 23 }

Shard A

Shard B

Shard C

$\text{minKey} \leq X < 10$

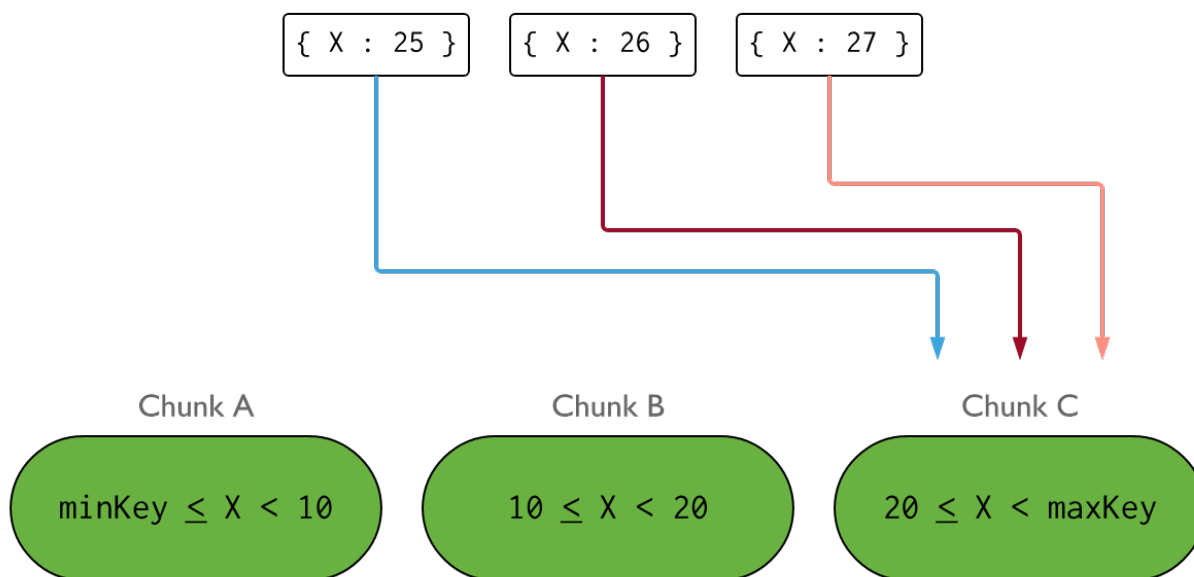
$10 \leq X < 20$

$20 \leq X < \text{maxKey}$



MongoDB——选择片键 (2)

- 可能不是好的片键的例子
 - 单调片键（升序或降序）
 - 例如使用类似时间戳的字段作为片键
 - 优点是近段时间需要访问的数据在内存中的概率更大
 - 问题是导致新插入数据总是在同一个片键上，单个分片压力过大



MongoDB——选择片键（3）

- 可能不是好的片键的例子
 - 随机片键
 - 选择取值随机的字段作为片键
 - 开始效果不错，但是随着数据量增大，效果变差
 - 考虑当需要对分片进行平衡时，会将一个分片的一些块放到另一些分片上
 - 结果是：每个分片上随机分布着数据，可能需要放在内存中的常用数据也随机分布在各个分片中，使得磁盘压力变大，导致速度慢
- 好片键
 - 具备良好的数据局部性特征——高效查询
 - 相关数据会保持在同一个局部内，避免内存数据高频率切换
 - 不会产生数据热点——高效集群
 - 数据的插入、修改操作均匀的分布在多个分片上，而不是集中在某一个分片



MongoDB——选择片键（4）

- 使用组合片键
 - 一个字段用于将数据分块
 - 一个字段用于保持数据局部性特征
- 选择片键应该思考的问题
 - 写操作是怎样的？需要插入的文档类型和大小？
 - 系统写入数据的频率如何？
 - 哪些字段是随机值，哪些是增长的？
 - 读操作是怎样的？经常访问的数据有哪些？
 - 系统读取数据的频率如何？
 - 数据有索引吗？是否需要索引？
 - 数据总量有多少？

使用 7.0 中的分片键分析器查找分片键

从 7.0 开始，MongoDB 让选择分片键变得更容易。您可以使用 `analyzeShardKey` 计算评估未分片或分片集合的分片键的指标。指标基于采样查询，允许您根据数据选择分片键。

启用查询采样

要分析分片键，必须对目标 collection 启用查询采样。有关详细信息，请参阅：

- `configureQueryAnalyzer` 数据库命令
- `db.collection.configureQueryAnalyzer` Shell 方法

分片键分析命令

要分析分片键，请参阅：

- `analyzeShardKey` 数据库命令
- `db.collection.analyzeShardKey()` Shell 方法



目录

- 分布式存储系统
 - 数据和系统类型
 - 分布式文件系统
 - 从单机到分布式—具体如何分布式
- 案例1：区块链
- 案例2：MongoDB
- 案例3：Cassandra
- 案例3：Ceph



- Cassandra是一个女性名字,源自希腊神话中的美丽女神卡珊德拉(Cassandra)。在希腊语中,Cassandra意为“发光者”或“闪耀者”。
- Cassandra这个名字给人的印象是有组织能力、善于表达、积极向上、自信,适合团队工作,并能很好地组织。

Open Source NoSQL Database

Manage massive amounts of data, fast, without losing sleep

Solving Big Data Challenges for Enterprise Application Performance Management

• for s

nance

• {

re

• {

Tilman Rabl
Middleware Systems
Research Group
University of Toronto, Canada
tilmann@msrg.utoronto.ca

Mohammad Sadoghi
Middleware Systems
Research Group
University of Toronto, Canada
mo@msrg.utoronto.ca

Hans-Arno Jacobsen
Middleware Systems
Research Group
University of Toronto, Canada
arno@msrg.utoronto.ca

re

• {

Sergio Gómez-Villamor
DAMA-UPC
Universitat Politècnica de
Catalunya, Spain
sgomez@ac.upc.edu

Victor Muntés-Mulero
CA Labs Europe
Barcelona, Spain
victor.muntes@ca.com

Serge Mankovskii
CA Labs
San Francisco, USA
serge.mankovskii@ca.com

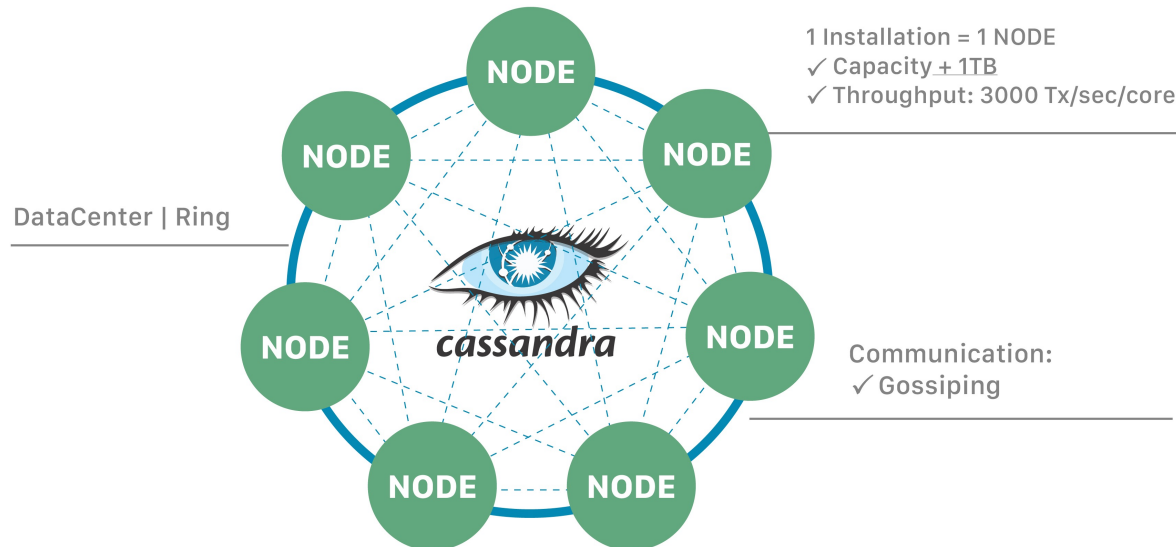
and



Cassandra——架构

- Cassandra can (and usually does) have **multiple nodes**.
 - A node represents a single instance of Cassandra.
 - These nodes communicate with one another through a protocol called **gossip**, which is a process of computer peer-to-peer communication.
- Cassandra also has a **masterless architecture**.
 - Any node in the database can provide the exact same functionality as any other node – contributing to Cassandra's robustness and resilience.

ApacheCassandra™ = NoSQL Distributed Database



Cassandra——协议

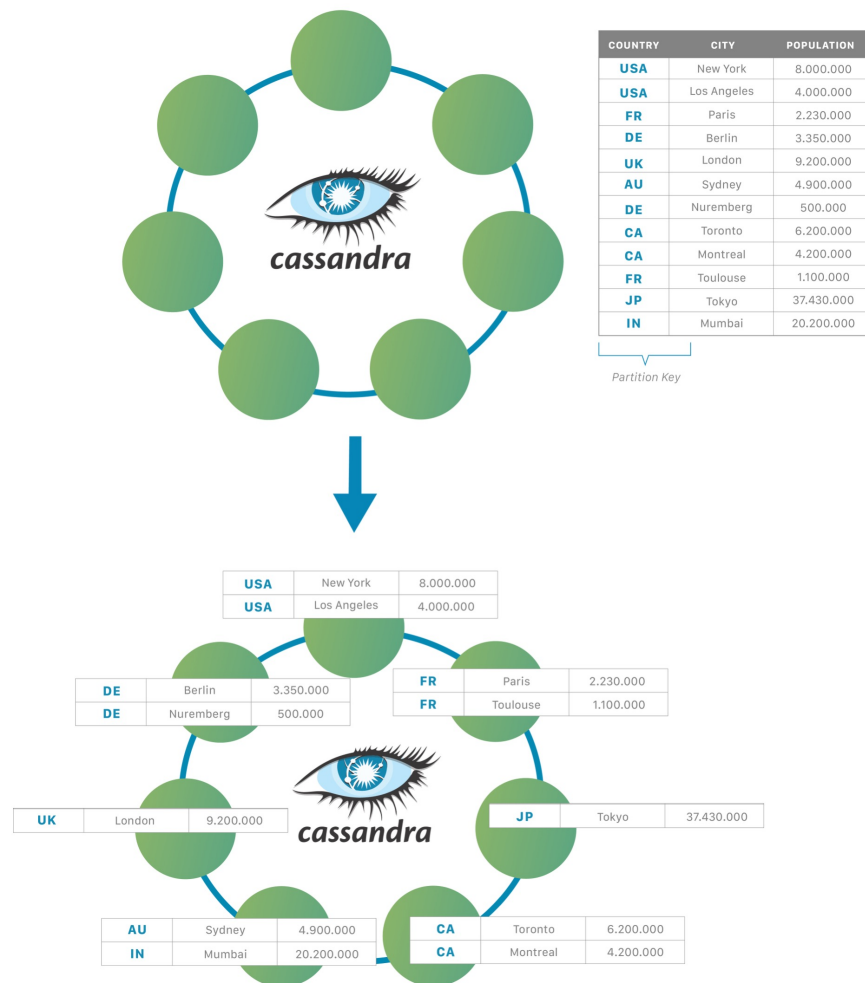
- **Gossip协议**是一个通信协议，一种传播消息的方式，灵感来自于：瘟疫、社交网络等，在分布式系统中被广泛使用，主要通过 Gossip 协议来保证网络中所有节点的数据一致性。
 - 利用一种随机的方式将信息传播到整个网络汇总，并在一定时间内，使得系统内的所有节点数据一致。
 - 是一种去中心化的分布式一致性协议，保证了数据在集群中的传播和状态一致性。
- **可扩展性**：一般只需要 $O(\log N)$ 轮就可以将信息传播到所有的节点，其中 N 代表节点的个数。每个节点仅发送固定数量的消息，在数据传输时，由于交叉重复传播的性质，节点并不需要等待消息的反馈，即使消息传输失败，也可以通过其他节点将信息传递给失败节点，系统可以轻松扩展到数百万个进程。
- **容错性**：任何节点的宕机或重启，都不影响整个协议的运行。
- **异步性**：其他分布式一致性协议，如Raft、ZAB协议，都需要等待节点反馈。
- **健壮性**：集群中节点都是对等的，任何节点都可以随时加入或离开。
- **最终一致性**：实现信息指数级传播，在有限的时间内能让所有节点有最新数据。



Cassandra——扩展

- It enables developers to **scale their databases dynamically**, using off-the-shelf hardware, with no downtime. You can expand when you need to – and also shrink, if the application requirements suggest that path.

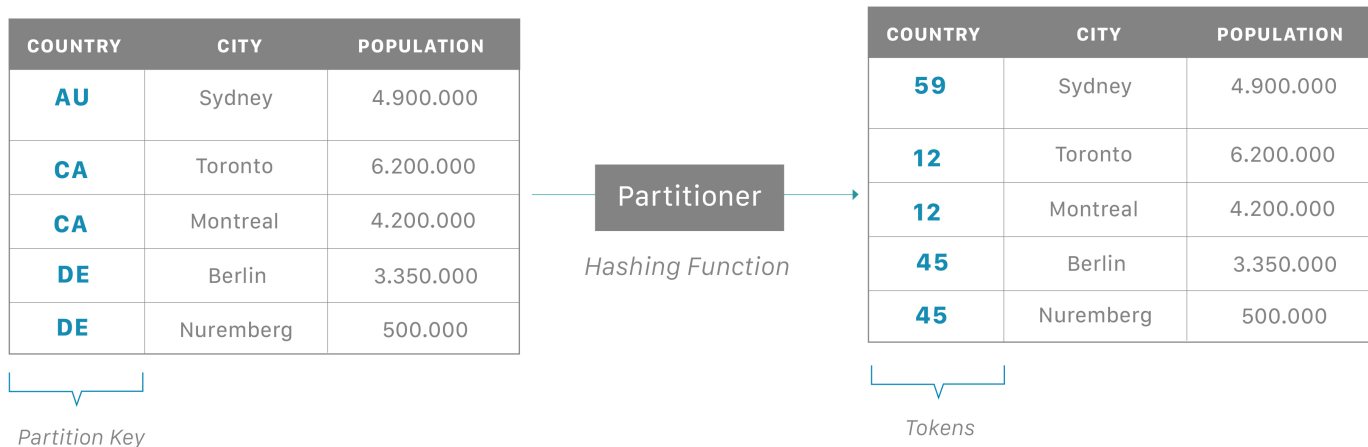
- 关系型数据库的扩容是很复杂的
 - 可以简单的对单台机器增强能力，昂贵
 - 可以使用集群但需要考虑数据依赖，复杂
- Cassandra以节点扩容，简单、便宜设备



Cassandra——分片

- Partition

- In Cassandra, the data itself is automatically distributed, with (positive) performance consequences. It accomplishes this using partitions.
- Each node owns a particular set of tokens, and Cassandra distributes data based on the ranges of these tokens across the cluster. The **partition key** is responsible for distributing data among nodes and is important for determining data locality.

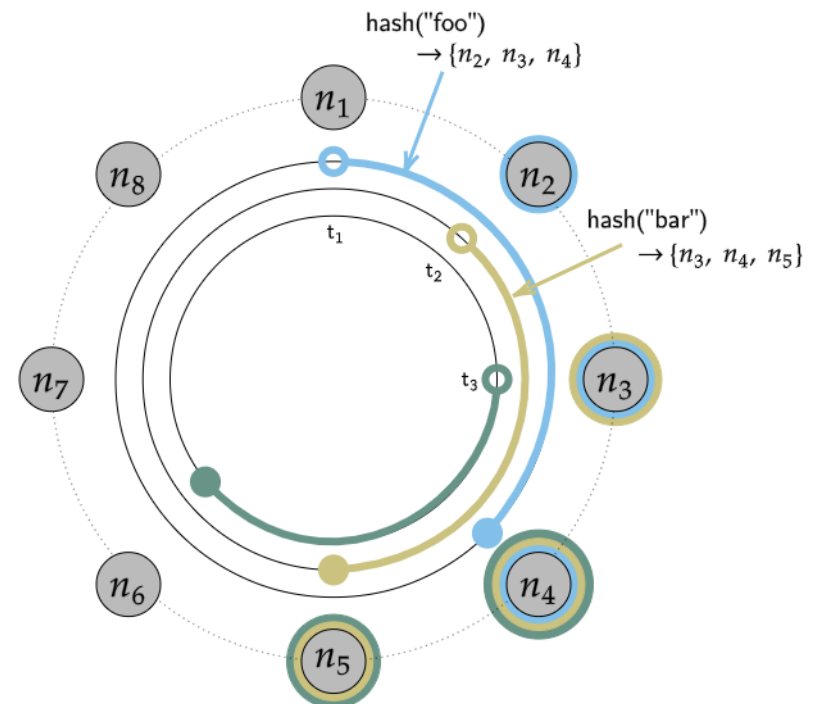
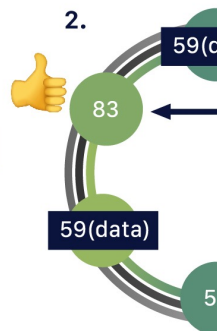
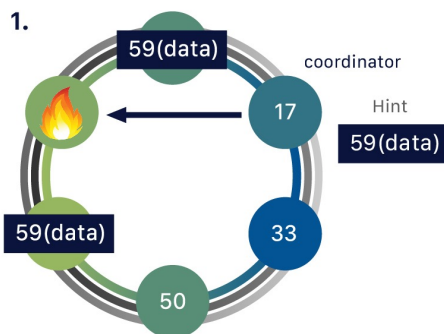
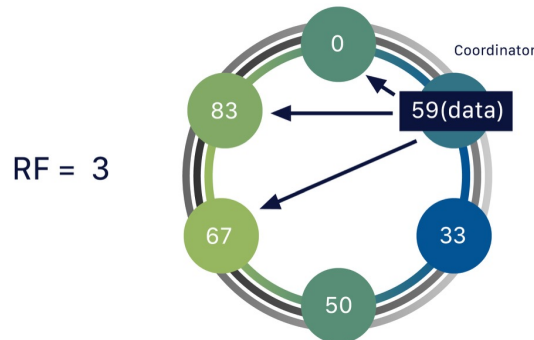


Cassandra——冗余

- Replication——一致性哈希
- One piece of data can be replicated to multiple (replica) nodes, ensuring reliability and fault tolerance. Cassandra supports the notion of a replication factor (RF), which describes how many copies of your data should exist in the database.

$\text{token}(n_1) = t_1$
 $\text{token}(n_2) = t_2$
 $\text{token}(n_3) = t_3$
...

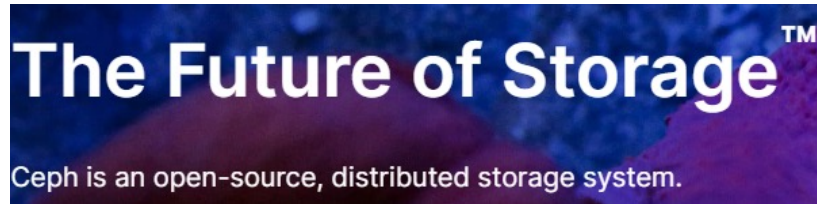
$\text{range}(t_1, t_2] \rightarrow \{n_2, n_3, n_4\}$
 $\text{range}(t_2, t_3] \rightarrow \{n_3, n_4, n_5\}$
 $\text{range}(t_3, t_4] \rightarrow \{n_4, n_5, n_6\}$
...



目录

- 分布式存储系统
 - 数据和系统类型
 - 分布式文件系统
 - 从单机到分布式—具体如何分布式
- 案例1：区块链
- 案例2：MongoDB
- 案例3：Cassandra
- 案例3：Ceph





The Ceph Foundation believes that all storage problems should be solvable with **open-source software**.

Reliable and scalable storage designed for any organization

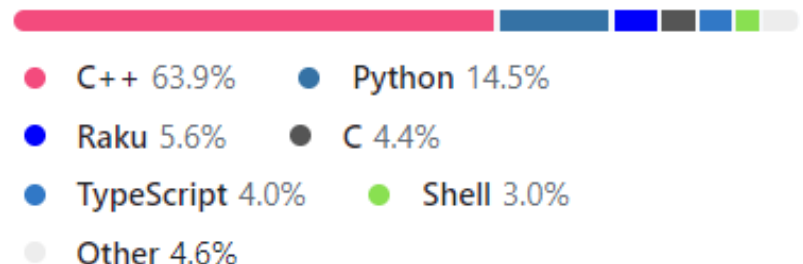
Use Ceph to transform your storage infrastructure. Ceph provides a unified storage service with object, block, and file interfaces from a single cluster built from commodity hardware components.

Contributors 1,320

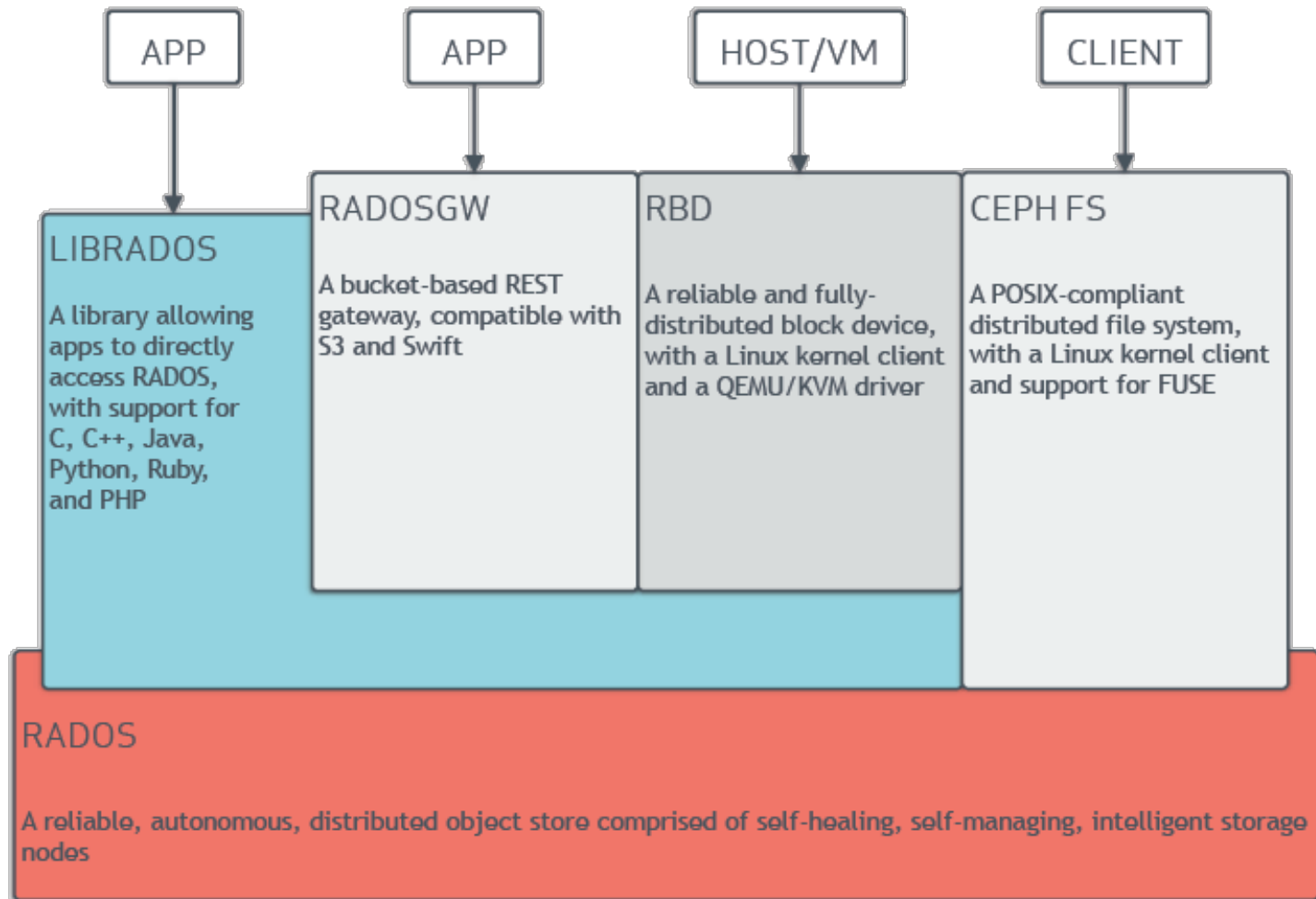


[+ 1,306 contributors](#)

Languages



Ceph——架构



Ceph—CSC

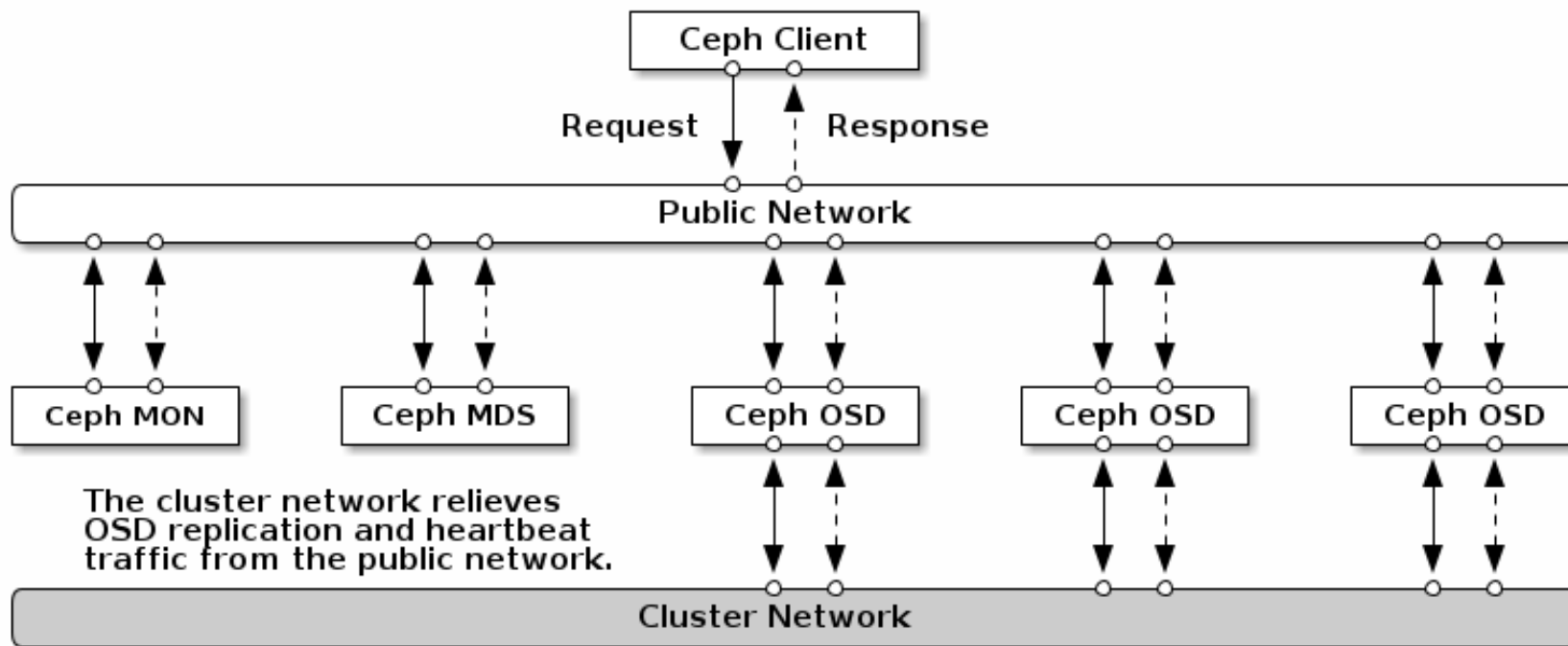
- The **Ceph Storage Cluster** is the foundation for all Ceph deployments. Based upon **RADOS**, Ceph Storage Clusters consist of several types of daemons:
 - a Ceph OSD Daemon (OSD) stores data as objects on a storage node
 - a Ceph Monitor (MON) maintains a master copy of the cluster map.
 - a Ceph Manager manager daemon
- A Ceph Storage Cluster might contain thousands of storage nodes. A minimal system has at least one Ceph Monitor and two Ceph OSD Daemons for data replication.
- The Ceph File System, Ceph Object Storage and Ceph Block Devices read data from and write data to the Ceph Storage Cluster.

OSD BACK ENDS

There are two ways that OSDs manage the data they store. As of the Luminous 12.2.z release, the default (and recommended) back end is BlueStore. Prior to the Luminous release, the default (and only) back end was Filestore.



Ceph——网络



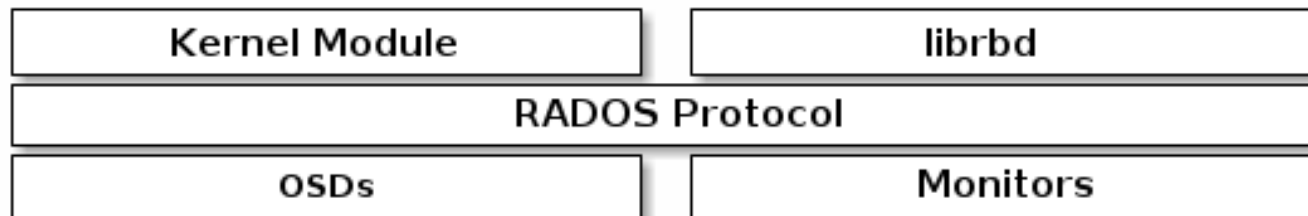
Ceph——对象存储

- Ceph Object Storage supports two interfaces:
 - **S3-compatible**: Provides object storage functionality with an interface that is compatible with a large subset of the **Amazon S3** RESTful API.
 - **Swift-compatible**: Provides object storage functionality with an interface that is compatible with a large subset of the **OpenStack Swift API**.
- Ceph Object Storage uses the Ceph Object Gateway daemon, an HTTP server designed to interact with a Ceph Storage Cluster.
- The Ceph Object Gateway has its own user management.
- The S3 API and the Swift API **share a common namespace**, which means that it is possible to write data to a Ceph Storage Cluster with one API and then retrieve that data with the other API.



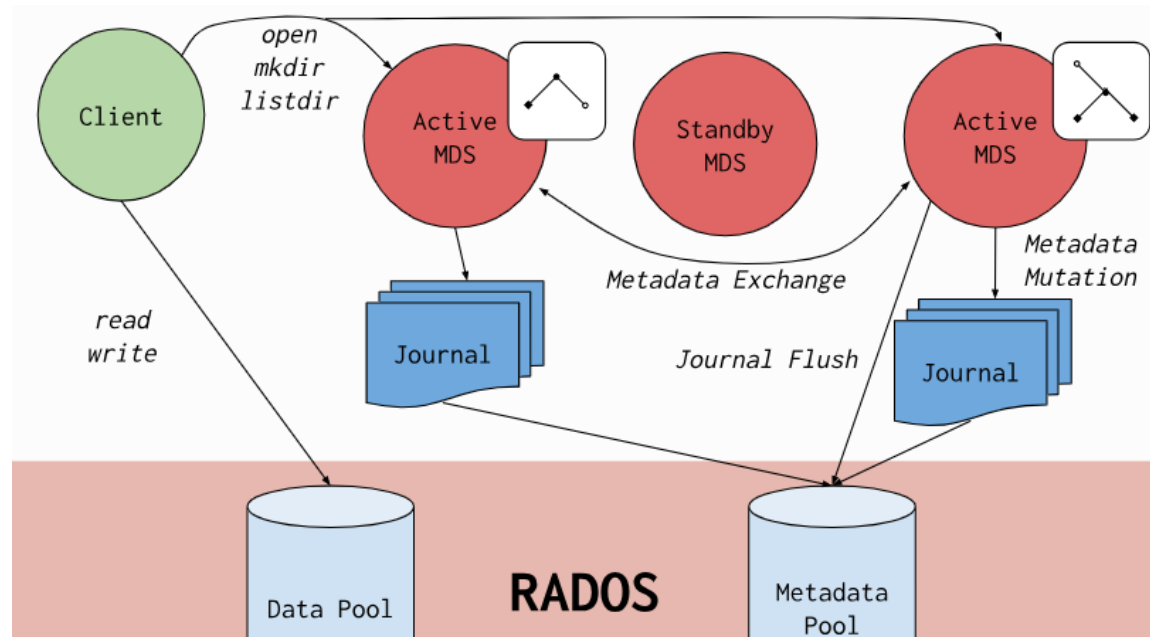
Ceph——块存储

- A block is a sequence of bytes (often 512).
- Block-based storage interfaces are a mature and common way to store data on media including HDDs, SSDs, CDs, floppy disks, and even tape.
- The ubiquity of block device interfaces is a perfect fit for interacting with mass data storage including Ceph.
- Ceph block devices are thin-provisioned, resizable, and store data striped over multiple OSDs.
- Ceph block devices leverage RADOS capabilities including snapshotting, replication and strong consistency. Ceph block storage clients communicate with Ceph clusters through **kernel modules** or the **librbd** library.

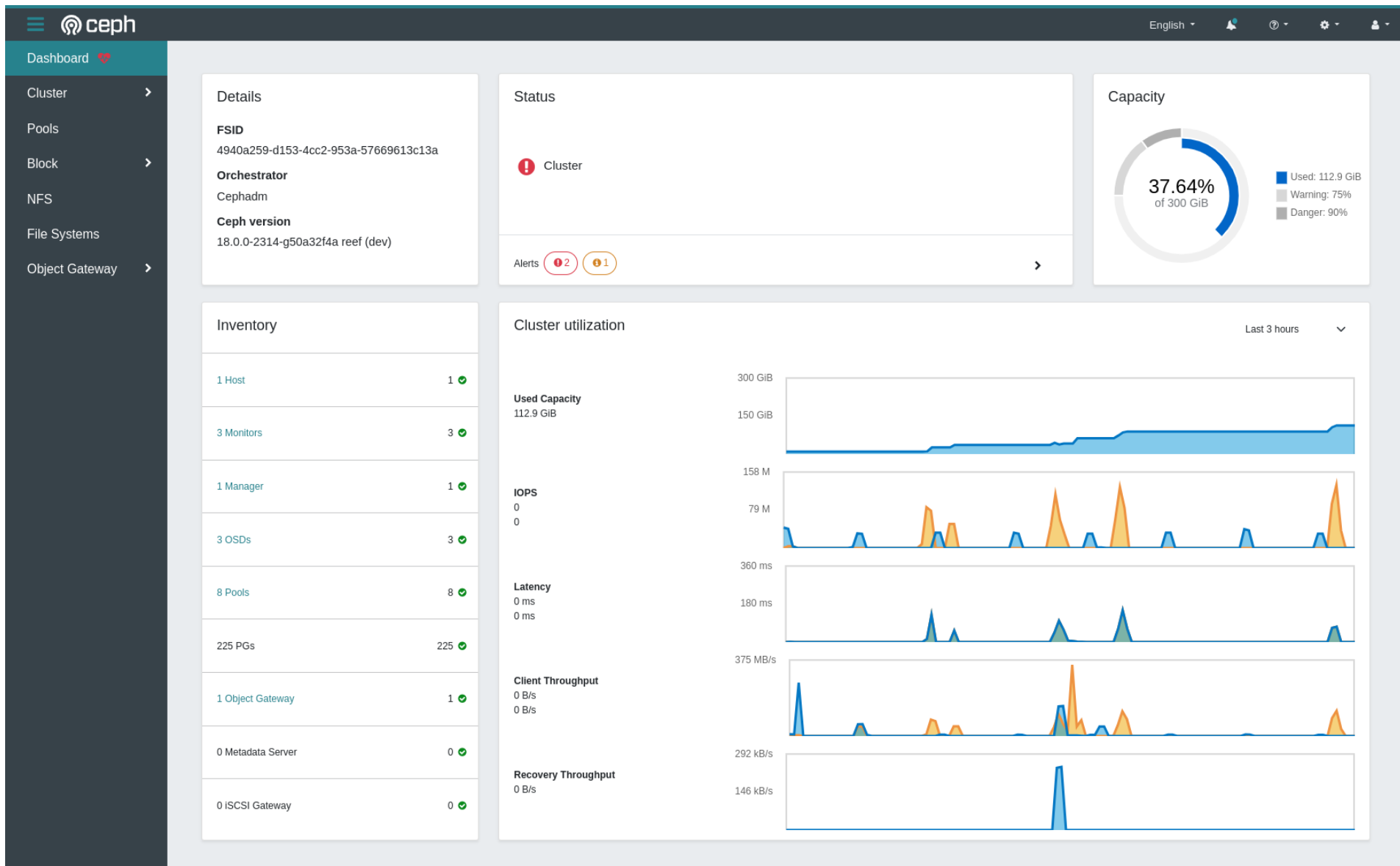


Ceph——文件存储

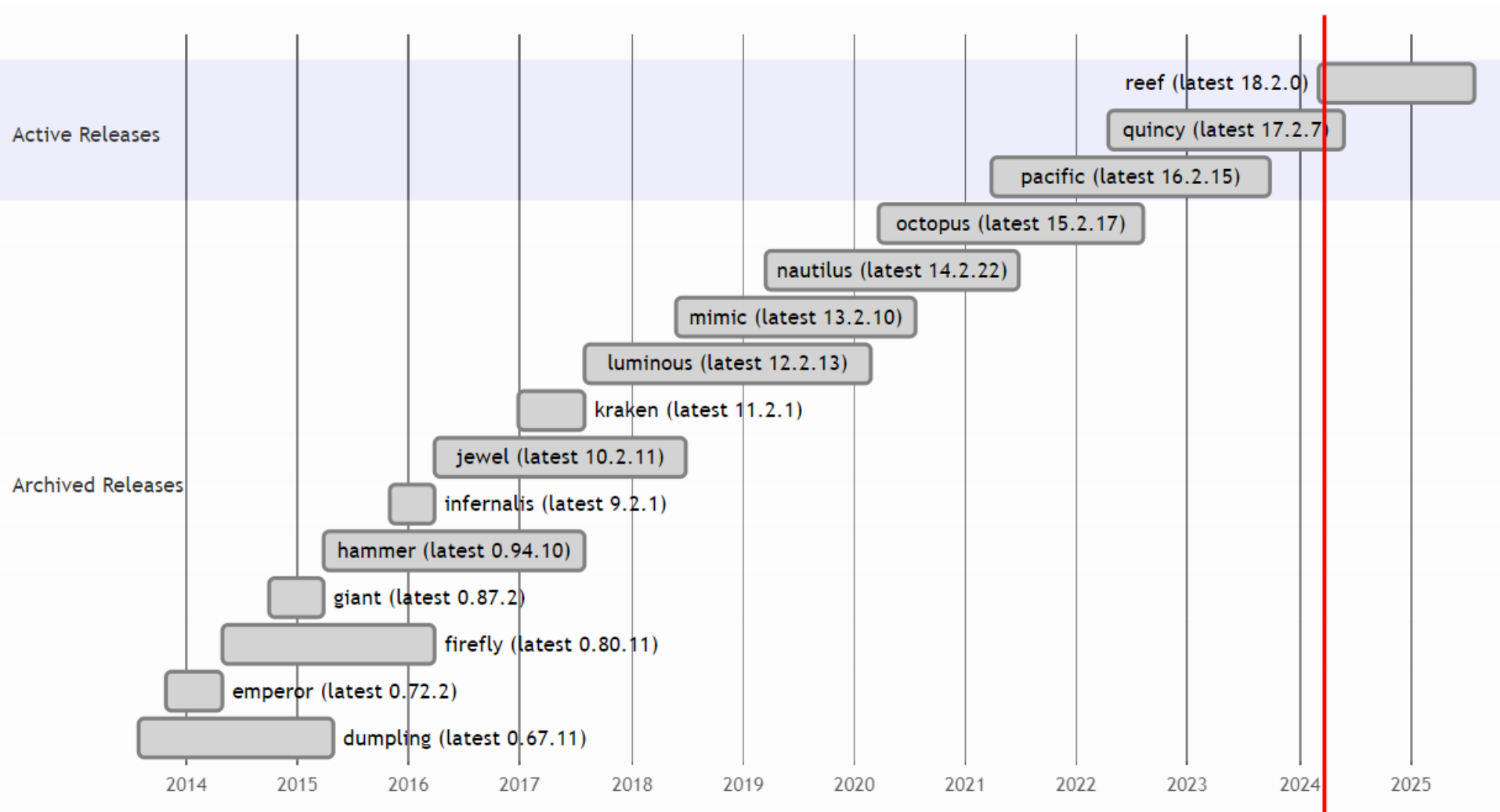
- The Ceph File System, or CephFS, is a POSIX-compliant file system built on top of Ceph' s distributed object store, RADOS.
- CephFS provide a SOTA, multi-use, highly available, and performant file store for a variety of applications, including **shared home directories**, **HPC scratch space**, and **distributed workflow shared storage**.
- A resizable cluster of Metadata Servers, or MDS.
- Clients of the file system have direct access to RADOS for reading and writing file data blocks.



Ceph—Dashboard



Ceph—Releases



谢 谢



南京大學
NANJING UNIVERSITY